

به نام خدا

گزارش چهارم آزمایشگاه سیستم عامل

سید علیرضا میرشفیعی - ۸۱۰۱۰۱۵۳۲

محمد تقی زاده - ۸۱۰۱۰۲۵۸۹

محمد صدرا عباسی - ۸۱۰۱۰۱۴۶۹

جواب سوالات تئوری به تفکیک بخش هاش:

بخش اول: تحلیل هسته و قفل‌های پیشرفته

۱.

مسیر کنترلی (Control Path) به جریان اجرای دستورالعمل‌ها توسط پردازنده گفته می‌شود که وضعیت ثابت‌های پردازنده را در هر لحظه تعیین می‌کند. تفاوت اصلی میان مسیرهای کنترلی در منشأ ایجاد آن‌هاست: مسیرهای ایجاد شده توسط «فراخوانی سیستمی» (Syscall) ناشی از درخواست ارادی و همگام برنامه کاربر برای دریافت سرویس از کرنل هستند، در حالی که مسیرهای ناشی از «وقفه» (Interrupt) به دلیل رخداد‌های سخت‌افزاری خارجی (مانند دیسک یا تایمر) و به صورت ناهمگام با اجرای جاری ایجاد می‌شوند. «استثنا» (Exception) نیز رویدادی است که در اثر خطای اجرای دستورالعمل جاری (مانند تقسیم بر صفر) یا شرایط خاص توسط خود پردازنده تولید می‌شود.

۲.

اصطلاح Reentrant Kernel یا «هسته با قابلیت ورود مجدد» به معنای کرنلی است که در آن چندین پردازنده یا مسیر اجرایی می‌توانند به صورت هم‌زمان (در چند هسته‌ای) یا شبه هم‌زمان (در تک هسته‌ای) در فضای کرنل اجرا شوند، بدون اینکه تداخلی در داده‌های سیستم ایجاد کنند. در سیستم عامل XV6، ممکن است یک پردازنده در حال اجرای یک Syscall باشد و تغییراتی را در ساختارهای کرنل آغاز کرده باشد؛ در همین حین، یک وقفه دیسک رخ می‌دهد و پردازنده مجبور می‌شود اجرای جاری را متوقف کرده و مجدداً وارد کد هسته شود تا هندلر وقفه را اجرا کند. در این حالت، اگر مکانیزم‌های همگام‌سازی وجود نداشته باشد، خطر جدی خرابی داده‌های کرنل وجود دارد، زیرا ممکن است هر دو مسیر تلاش کنند یک ساختار داده مشترک را تغییر دهند.

۳.

تفاوت بنیادین میان بستر پردازنده (Process Context) و بستر وقفه (Interrupt Context) در مالکیت و قابلیت‌های آن‌هاست. بستر پردازنده زمانی است که هسته به نمایندگی از یک پردازنده خاص (مثلاً در پاسخ به یک System Call) اجرا می‌شود و به حافظه مجازی و ساختار proc آن پردازنده دسترسی دارد. در مقابل، بستر وقفه صرفاً پاسخ به یک رویداد سخت‌افزاری است و به هیچ پردازنده خاصی تعلق ندارد. تفاوت کلیدی این است که کدهای اجرا شده در بستر پردازنده می‌توانند به خواب روند (Sleep)، اما این عمل در بستر وقفه به دلیل نداشتن زمینه پردازشی مشخص برای زمان‌بندی مجدد، مجاز نیست.

۴.

کدی که در بستر وقفه اجرا می‌شود، مانند هندلر تایمر، به هیچ وجه نباید مسدود (Block) شده یا به خواب (Sleep) برود. دلیل این محدودیت آن است که هندلر وقفه یک پدازه زمان‌بندی شده نیست؛ سیستم‌عامل نمی‌داند چگونه و چه زمانی باید آن را بیدار کند زیرا ساختار پدازه‌ای متناظر با آن در صف زمان‌بند وجود ندارد. اگر هندلر وقفه بخوابد، می‌تواند باعث قفل شدن سیستم (به ویژه در سیستم‌های تک هسته‌ای) یا از دست رفتن سایر وقفه‌های حیاتی شود که منتظر سرویس‌دهی هستند.

۵.

برای محافظت از داده‌های مشترک در بستر وقفه، معمولاً از ترکیب قفل چرخشی (Spinlock) و غیرفعال‌سازی وقفه‌ها (Disable Interrupts) استفاده می‌شود. استفاده از قفل‌های خواب (Sleep locks) در اینجا ممکن نیست چون هندلر وقفه طبق قانون قبلی اجازه خوابیدن ندارد. همچنین، اگر وقفه‌ها فعال بمانند و پردازنده در حال نگه داشتن یک قفل باشد، وقوع یک وقفه جدید می‌تواند باعث تلاش هندلر برای گرفتن همان قفل و ایجاد بن‌بست شود؛ بنابراین غیرفعال‌سازی وقفه‌ها نیز ضروری است.

۶.

سه رویکرد اصلی همگام‌سازی عبارتند از:

- **Spinlock (قفل چرخشی):** در این روش پردازنده در یک حلقه مداوم وضعیت قفل را چک می‌کند تا آزاد شود. مزیت آن سرعت بالا در انتظارهای کوتاه است، اما عیب اصلی آن اتلاف سیکل‌های پردازنده (Busy Waiting) است.
- **Sleep lock (قفل خواب):** در صورت اشغال بودن منبع، پردازنده جاری را به خواب می‌برد و پردازنده را آزاد می‌کند. مزیت آن جلوگیری از هدررفت پردازنده در انتظارهای طولانی است، اما سربار تغییر متن (Context Switch) را به همراه دارد.
- **Lock-free Programming:** روشی پیشرفته است که بدون استفاده از قفل و با دستورات اتمیک داده‌ها را همگام می‌کند. مزیت آن حذف بن‌بست (Deadlock) است، اما پیاده‌سازی آن بسیار پیچیده و دشوار است.

۷.

در تابع acquire، قبل از گرفتن قفل، وقفه‌ها غیرفعال (cli) می‌شوند تا از وقوع بن‌بست جلوگیری شود. فرض کنید وقفه‌ها باز باشند و یک پردازنده قفلی را بگیرد؛ سپس یک وقفه رخ دهد و پردازنده به سراغ اجرای هندلر وقفه برود. اگر هندلر وقفه بخواهد همان قفل را بگیرد، در یک حلقه بی‌نهایت منتظر آزاد شدن قفل می‌ماند. اما چون پردازنده در داخل هندلر گیر کرده است، کدی که قفل را در اختیار دارد (و قبل از وقفه اجرا می‌شد) هرگز

فرصت اجرا پیدا نمی‌کند تا قفل را آزاد کند؛ این وضعیت منجر به Deadlock قطعی در سیستم تک‌هسته‌ای می‌شود.

۸.

سیستم عامل XV6 به جای استفاده مستقیم از دستورات cli و sti، از توابع pushcli و popcli استفاده می‌کند تا مشکل نواحی بحرانی تودرتو (Nested Critical Sections) را حل کند. اگر از دستورات ساده استفاده شود، اولین فراخوانی sti در پایان یک تابع داخلی، وقفه‌ها را زودتر از موعد (در حالی که هنوز در تابع بیرونی و ناحیه بحرانی هستیم) فعال می‌کند. توابع pushcli و popcli با نگهداری یک شمارنده، تعداد دفعات غیرفعال‌سازی را می‌شمارند و تضمین می‌کنند که وقفه‌ها تنها زمانی مجدداً فعال شوند که تمام قفل‌های تودرتو آزاد شده باشند و شمارنده دقیقاً به صفر برسد.

بخش دوم: مقیاس‌پذیری داده‌های Per-CPU و پروفایلینگ قفل‌ها

۱.

زمانی که یک پردازنده یک قفل چرخشی (Spinlock) را در اختیار دارد، حتماً باید وقفه‌ها روی آن پردازنده غیرفعال باشند. این الزام حیاتی برای جلوگیری از ورود مجدد (Re-entry) به همان پردازنده توسط یک وقفه و تلاش هندلر آن برای گرفتن همان قفل است. اگر این اتفاق بیفتد، سیستم در یک حلقه بی‌پایان گرفتار می‌شود و هرگز نمی‌تواند قفل را آزاد کند.

۲.

اگر وقفه‌ها فعال بمانند و هندلر وقفه (Interrupt Handler) بخواهد همان قفلی را بگیرد که پردازنده پیش از وقوع وقفه در اختیار داشته است، نوعی از بن‌بست (Deadlock) رخ می‌دهد. در این سناریو، هندلر وقفه در حلقه acquire می‌چرخد و منتظر آزاد شدن قفل می‌ماند (Spinning)، در حالی که پروسه‌ای که قفل را در دست دارد، توسط همین هندلر متوقف شده (Preempted) و هرگز فرصت اجرا پیدا نمی‌کند تا قفل را آزاد کند.

۳.

وقتی چندین هسته پردازنده بخواهند به طور مداوم یک متغیر مشترک سراسری را تغییر دهند، پدیده‌ای رخ می‌دهد که باعث کندی شدید سیستم می‌شود. پروتکل‌های همگام‌سازی حافظه نهان مانند MESI در این شرایط مجبور می‌شوند با هر بار نوشتن یک هسته، کپی‌های آن متغیر در کش سایر هسته‌ها را نامعتبر (Invalidate) کنند. این فرآیند باعث ایجاد ترافیک سنگین روی گذرگاه (Bus) و توقف پردازنده‌ها برای همگام‌سازی مداوم داده‌ها می‌شود.

۴.

استفاده از متغیرهای محلی (Per-CPU) مشکل رقابت بر سر کش را تا حد زیادی کاهش می‌دهد. با اختصاص دادن شمارنده‌های جداگانه به هر پردازنده، هر هسته تنها متغیر موجود در کش خود را تغییر می‌دهد. در نتیجه، خط کش (Cache Line) مربوط به آن متغیر تنها در کش همان هسته تغییر می‌کند و نیازی به ارسال پیام‌های Invalidation به سایر هسته‌ها نیست، که این امر تعداد خطاهای کش و ترافیک گذرگاه را به شدت کم می‌کند.

۵.

تفاوت اصلی بین Spinlock (دسته اول توابع در Xv6) و Sleeplock (دسته دوم توابع) در نحوه مدیریت انتظار است. Spinlock باعث انتظار مشغول (Busy Waiting) می‌شود و پردازنده را درگیر نگه می‌دارد تا قفل آزاد شود. در مقابل، Sleeplock در صورت اشغال بودن قفل، پردازنده را آزاد کرده و پروسه جاری را به حالت خواب می‌برد تا زمان‌بند (Scheduler) بتواند پردازنده را به پروسس دیگری واگذار کند (Context Switch).